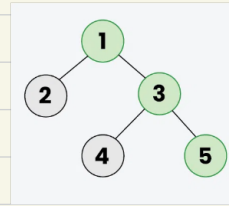
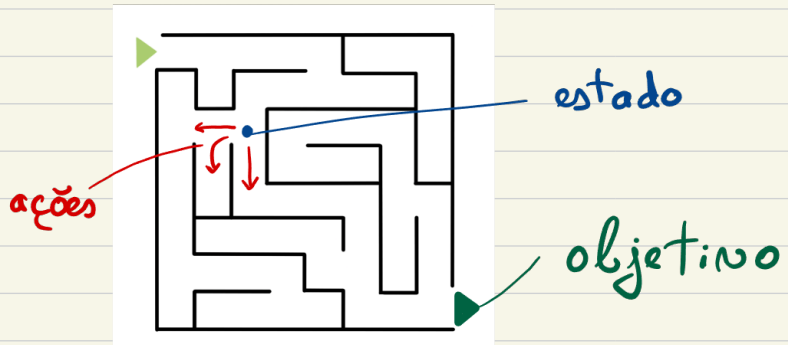


II Pesquisa em árvore



Tree Search - encontrar o caminho desde o estado inicial até a um estado objetivo



Conceitos

State - representação de uma situação possível no problema

Action - operação que transforma um estado montado

Result - é um novo estado que se obtém ao aplicar uma ação a um estado

Cost - cada ação pode ter um custo associado

Goal - estado final desejado

Diferentes tipos de classes:

(1) Search Domain - define o dominio do problema, deve ter metodos como: actions(state), result(state, action), cost(state, action)

💡 Podemos pensar como o "manual de regras do jogo"

(2) Search Problem() - problemas concretos a resolver dentro de um determinado dominio

(3) SearchNode() - é um nó na arvore de pesquisa, representando um estado mais a informação de como lá chegamos

state
parent
depth
cost
⋮

(4) SearchTree() - é a árvore de pesquisa propriamente dita
Ela gerá o processo de procura de uma solução

```
# constructor
def __init__(self, problem, strategy='breadth'):
    self.problem = problem
    root = SearchNode(problem.initial, None)
    self.open_nodes = [root]
    self.strategy = strategy
    self.solution = None
    self.length = None
    self.terminals = 0
    self.non_terminals = 0
```

lista de nós a explorar

Contadores

guardar a profundidade

guardar a solução

nó inicial (sem Pai)

```
# obter o caminho (sequencia de estados) da raiz ate um nó
def get_path(self, node):
    if node.parent == None:
        return [node.state]
    path = self.get_path(node.parent)
    path += [node.state]
    return(path)
```

Caso Base

Vai adicionar o estado atual

reconstrói o caminho desde a raiz até ao nó, recursivamente ⚠

O método search() é o **motor da pesquisa** na árvore

```
1 def search(self, limit=None):
2     while self.open_nodes != []:
3         node = self.open_nodes.pop(0)
4         if self.problem.goal_test(node.state):
5             self.solution = node
6             self.length = node.depth
7             self.terminals = len(self.open_nodes) + 1 # nós que ficaram por expandir em open_nodes são folhas
8             return self.get_path(node)
9         self.non_terminals += 1 # não é solução, logo está a ser expandido
10        lnewnodes = []
11        for a in self.problem.domain.actions(node.state):
12            newstate = self.problem.domain.result(node.state, a)
13            if newstate not in self.get_path(node):
14                new_cost = node.cost + self.problem.domain.cost(node.state, a) # custo acumulado
15                newnode = SearchNode(newstate, node, new_cost)
16                if limit is None or newnode.depth <= limit:
17                    lnewnodes.append(newnode)
18        self.add_to_open(lnewnodes)
19    return None
```

```
1 def add_to_open(self, lnewnodes):
2     if self.strategy == 'breadth':
3         self.open_nodes.extend(lnewnodes)
4     elif self.strategy == 'depth':
5         self.open_nodes[:0] = lnewnodes
6     elif self.strategy == 'uniform':
7         self.open_nodes.extend(lnewnodes)
8         self.open_nodes.sort(key=lambda node: node.cost)
9     pass
```

→ Depende da **estratégia**
que se escolhe no início